Prompt Phrasing and String Reversal Accuracy in GPT-3.5 Turbo

Gabriel Lopes
UC San Diego

Which prompt phrasing leads to the highest accuracy for string reversal tasks in ChatGPT?

1. Introduction

Large Language Models (LLMs), such as OpenAI's ChatGPT, have become widely used for tasks ranging from natural language generation to reasoning and information retrieval. These models are trained on vast amounts of text data and can generate fluent responses across a wide range of contexts.

However, their performance often depends heavily on how input is phrased. A concept known as prompt engineering. Prompt engineering refers to the practice of designing input text strategically in order to influence the model’s output. Subtle differences in prompt wording can significantly affect task performance, especially on tasks that require precise symbolic manipulation rather than just surface-level language prediction.

In this project, I explore how different prompt formulations affect ChatGPT's ability to reverse strings accurately. A task that, while simple for humans, can be deceptively difficult for language models.

2. Background and Motivation

While LLMs like ChatGPT are good at generating coherent and plausible-sounding text, they are not very strong with exact symbolic tasks. A good example of an exact symbolic task is string reversal, which poses a challenge to the symbol manipulation abilities of a model on a character-by-character basis, which is out of the realm of language prediction and challenges deeper computational power.

There are a number of reasons why this task is useful in studying LLMs:

- It has easy right/wrong answers, so it is simple to assess.
- It needs form-based reasoning rather than semantic knowledge.
- It reveals how well LLMs perform on non-naturalistic, structured tasks.

More recent arguments in the literature challenge how much LLMs actually "know" facts such as these. For instance, Mitchell (2023) argues that LLMs are good at some reasoning tasks not because they have reasoning ability, but due to pretraining memorization and pattern-matching. If given novel versions of familiar tasks (e.g., counterfactuals), LLMs perform very badly, which suggests poor generalization and abstract reasoning ability.

Similarly, Bender and Koller (2020) emphasize that LLMs are trained on form alone, without any grounding in meaning or communicative intent. In other words, even when models do well, they could simply be repeating statistical patterns across text rather than necessarily knowing what words and phrases mean. This shortcoming is especially relevant on symbolic tasks like reversing a string, where one prevails not on the basis of fluency but on strict form-based operations.

Collectively, these positions suggest that the wording of a prompt, the way a question is asked, can disproportionately influence LLM performance, especially where the task includes clear rules or transformations. A slight change in wording can determine whether the model acquires the task properly or simply imitates an exact pattern from its training set.

3. Methods

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
import time
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```

3.1 Task Design

To evaluate ChatGPT's sensitivity to prompt phrasing, I selected five words of varying complexity: `"pan"`, `"cheese"`, `"cabbage"`, `"archetypal"`, and `"subsequently"`.

I designed four different prompt variants, ranging from direct instructions to more conversational styles:

- **Prompt A**: "Reverse the following string:"
- **Prompt B**: "Flip the letters in this word backwards:"
- **Prompt C**: "Write this word in reverse order:"
- **Prompt D**: "Can you tell me the reverse of this word?"

Each combination of prompt and word was tested **five times**, and performance was measured using a **binary accuracy metric**:
a response was considered correct only if it **exactly matched** the string reversed character-by-character.

```
prompts = {
    "Prompt A": "Reverse the following string: ",
    "Prompt B": "Flip the letters in this word backwards: ",
    "Prompt C": "Write this word in reverse order: ",
    "Prompt D": "Can you tell me the reverse of this word? "
}

strings = ["pan", "cheese", "cabbage", "archetypal", "subsequently"]
```

3.2 Model and Setup

To run the experiment, I used the **OpenAI GPT-3.5 Turbo** model via the official Python SDK in **Google Colab**.
Each prompt–word pair was run five times using:

- **OpenAI API (gpt-3.5-turbo)**
- **Google Colab as the coding environment**
- **Temperature = 0** to ensure consistent outputs
- **1-second delay** between API calls to avoid rate limits

Each output was collected and evaluated for correctness using a simple string comparison:

```
is_correct = result == correct_answer
```

```
def get_response(prompt, word):
    full_prompt = prompt + word + " Respond with only the reversed word."
    try:
        response = client.chat.completions.create(
            model="gpt-3.5-turbo",
            messages=[
                {"role": "user", "content": full_prompt}
            ],
            temperature=0
        )
        return response.choices[0].message.content.strip()
    except Exception as e:
        print(f"Error for input '{full_prompt}':", str(e))
        return f"ERROR: {str(e)}"
```

```
data = []

for prompt_label, prompt_text in prompts.items():
    for word in strings:
        correct_answer = word[::-1]
        for trial in range(5):
            result = get_response(prompt_text, word)
            is_correct = result == correct_answer
            data.append({
                "Prompt": prompt_label,
                "Word": word,
                "Response": result,
                "Correct Answer": correct_answer,
```

```
                "Is Correct": is_correct
            })
            time.sleep(1)
```

```
df = pd.DataFrame(data)
df
```

	Prompt	Word	Response	Correct Answer	Is Correct
0	Prompt A	pan	nap	nap	True
1	Prompt A	pan	nap	nap	True
2	Prompt A	pan	nap	nap	True
3	Prompt A	pan	nap	nap	True
4	Prompt A	pan	nap	nap	True
...	...	...	...	...	...
95	Prompt D	subsequently	ylatnerepus	yltneuqesbus	False
96	Prompt D	subsequently	ylatnubcesubs	yltneuqesbus	False
97	Prompt D	subsequently	ylatnerepus	yltneuqesbus	False
98	Prompt D	subsequently	ylatnubusnoces	yltneuqesbus	False
99	Prompt D	subsequently	ylatnerepus	yltneuqesbus	False

100 rows × 5 columns

4. Results

4.1 Overall Prompt Accuracy

The first analysis aggregated accuracy across all words to compare the performance of each prompt.
The chart below shows the average percentage of correct responses for each prompt formulation.

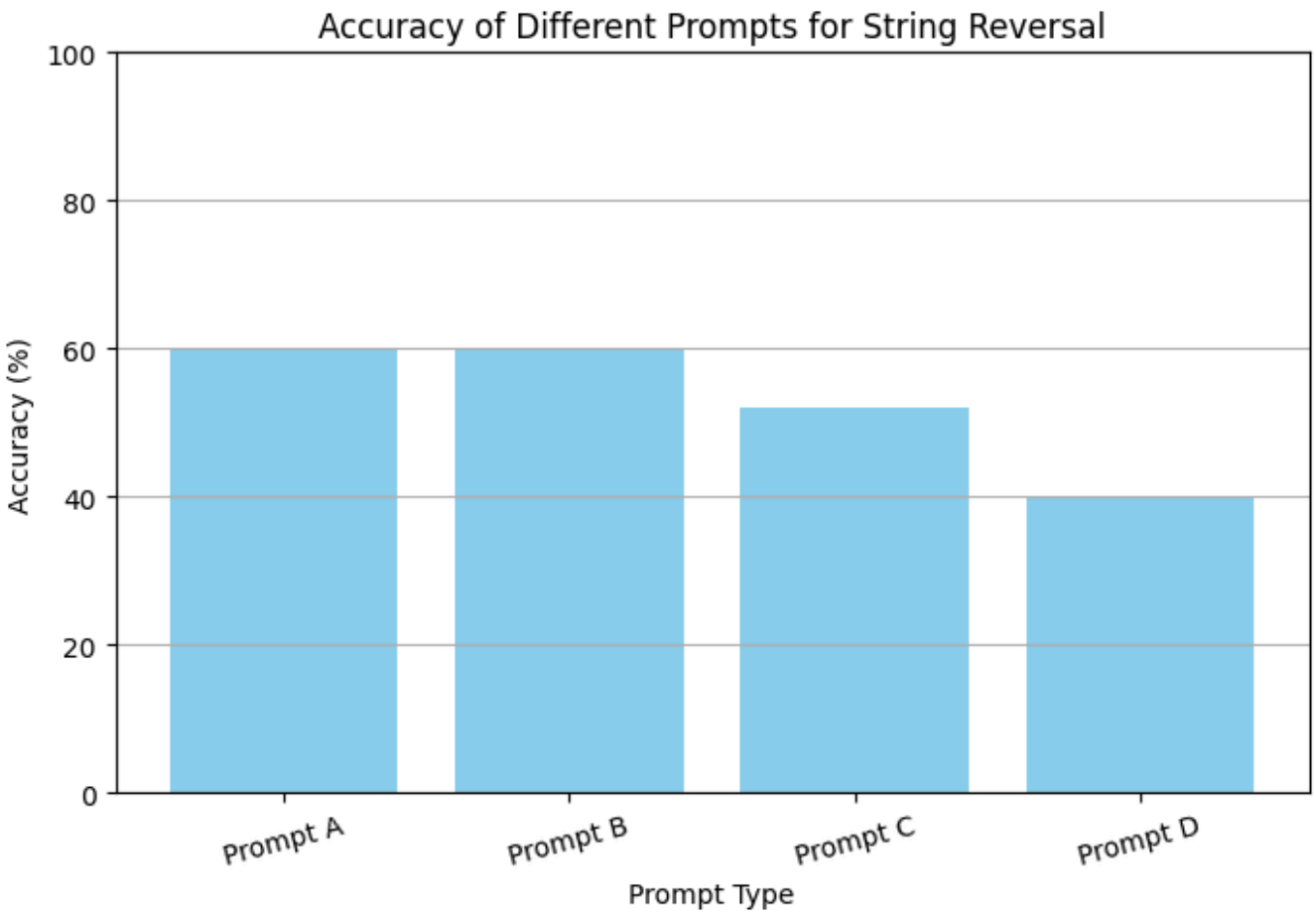
Findings:

- **Prompt A** and **Prompt B** had the highest accuracy (60%).
- **Prompt C** performed slightly worse (52%).
- **Prompt D**, the most conversational prompt, performed the worst (40%).

This suggests that more direct or imperative prompts led to higher success on symbolic tasks like string reversal.

```
accuracy_df = df.groupby("Prompt")["Is Correct"].mean().reset_index()
accuracy_df["Accuracy (%)"] = accuracy_df["Is Correct"] * 100
```

```
plt.figure(figsize=(8,5))
plt.bar(accuracy_df["Prompt"], accuracy_df["Accuracy (%)"], color='skyblue')
plt.ylim(0, 100)
plt.title("Accuracy of Different Prompts for String Reversal")
plt.ylabel("Accuracy (%)")
plt.xlabel("Prompt Type")
plt.xticks(rotation=15)
plt.grid(True, axis='y')
plt.show()
```



4.2 Accuracy by Prompt × Word

To explore whether certain prompts work better for specific words, I created a breakdown of accuracy by both prompt type and target word.

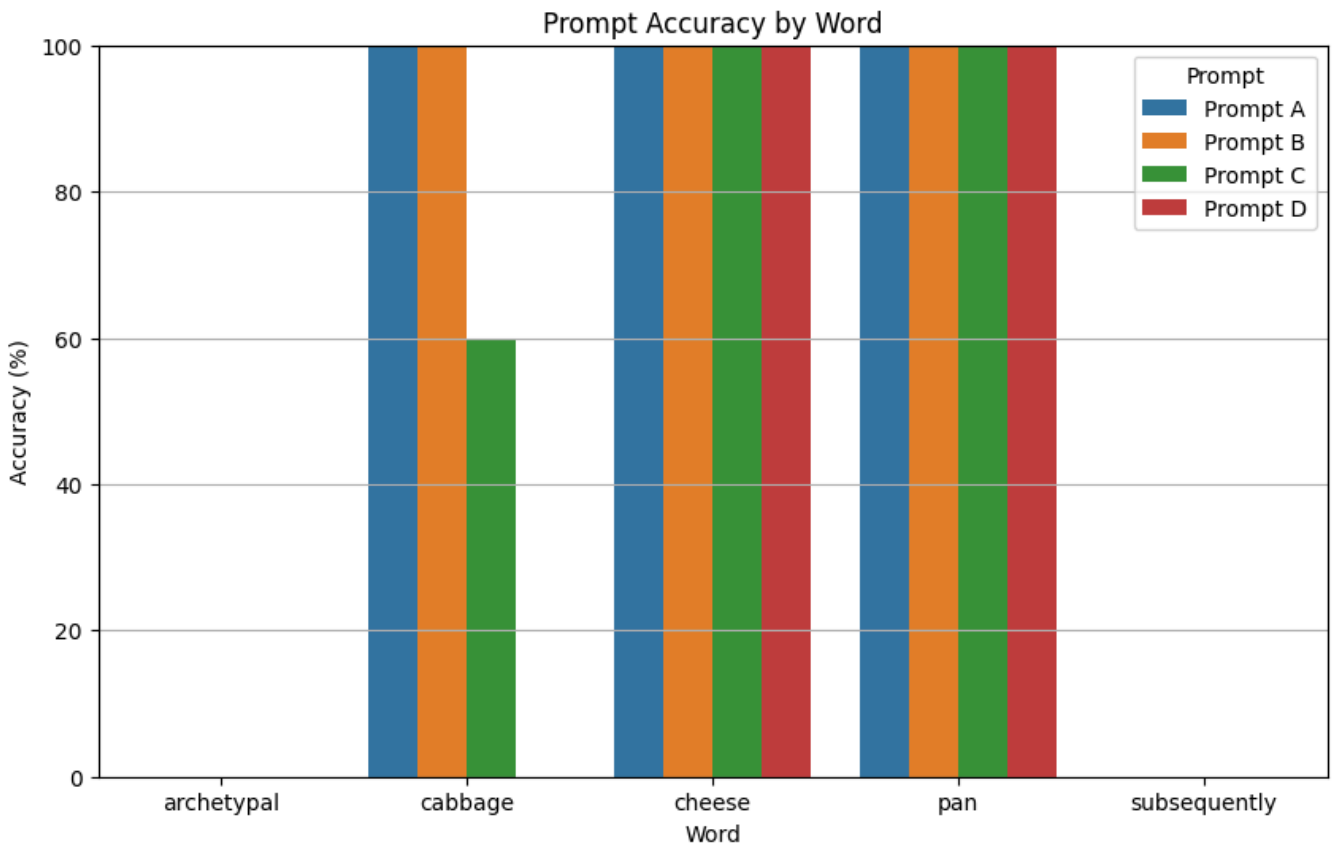
Observations:

- The words "pan", "cheese", and "cabbage" were reversed correctly most of the time across prompts A, B, and C.
- However, "archetypal" and "subsequently" had 0% accuracy for all prompt types, suggesting difficulty with longer or more complex words.
- Prompt C showed slightly lower performance on "cabbage" compared to A and B, getting 60% accuracy, potentially due to vague phrasing.

- Prompt D was the most inaccurate prompt, reversing "cabbage", "archetypal", and "subsequently" incorrectly every time.

This analysis reveals that task difficulty interacts with prompt clarity. Longer or unusual words may exceed the model's token-level reasoning ability unless the prompt is especially explicit.

```
word_accuracy = df.groupby(["Prompt", "Word"])["Is Correct"].mean().reset_index()
word_accuracy["Accuracy (%)"] = word_accuracy["Is Correct"] * 100
word_accuracy.sort_values(by=["Word", "Prompt"], inplace=True)
word_accuracy

plt.figure(figsize=(10, 6))
sns.barplot(data=word_accuracy, x="Word", y="Accuracy (%)", hue="Prompt")
plt.title("Prompt Accuracy by Word")
plt.ylim(0, 100)
plt.legend(title="Prompt")
plt.grid(axis='y')
plt.show()
```



4.3 Word Difficulty

To examine whether some words were inherently harder to reverse, I calculated the error rate per word, averaging across all prompts.

Results:

- "Archetypal" and "subsequently" had a 100% error rate.

- "Cabbage" had a moderate error rate (35%).
- "Pan" and "cheese" were always reversed correctly.

These results indicate a strong link between string complexity/length and performance. This supports prior findings that LLMs struggle with form-based generalization when tasks deviate from patterns commonly seen in training data.

```
word_difficulty = df.groupby("Word")["Is Correct"].mean().reset_index()
word_difficulty["Error Rate (%)"] = 100 - (word_difficulty["Is Correct"] * 100)
word_difficulty.sort_values(by="Error Rate (%)", ascending=False)
```

	Word	Is Correct	Error Rate (%)
0	archetypal	0.00	100.0
4	subsequently	0.00	100.0
1	cabbage	0.65	35.0
2	cheese	1.00	0.0
3	pan	1.00	0.0

4.4 Error Type Breakdown

Finally, I analyzed the types of mistakes the model made when it failed to reverse the string correctly.
Each incorrect response was categorized into:

- **Extra/missing letters**
- **Wrong order**
- **Off by 1 letter**
- **Other**

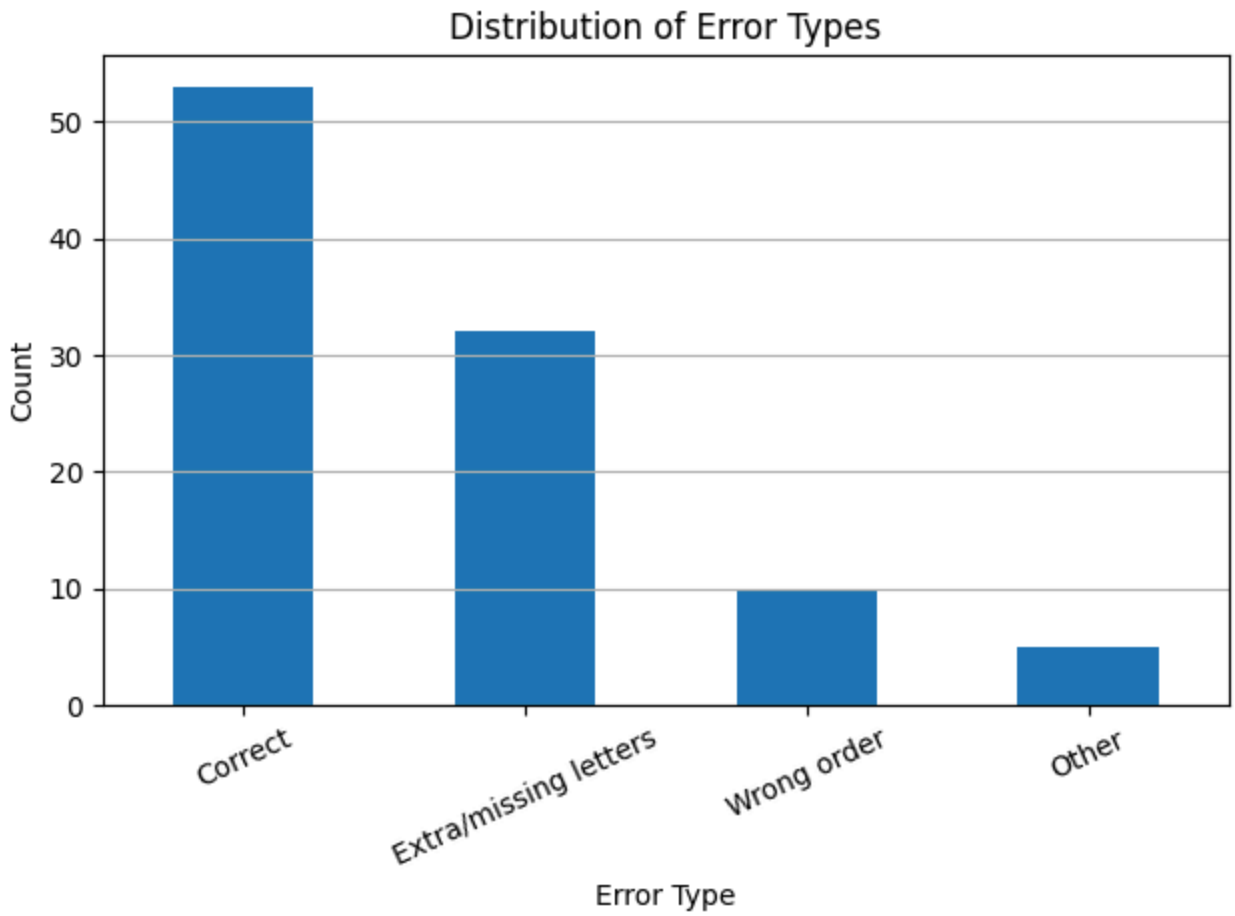
Summary:

- The most common error was "extra/missing letters", showing that ChatGPT often returned strings of the wrong length.
- Some responses had all the correct characters but in the wrong order, likely due to pattern matching instead of true reversal.
- Interestingly, no errors were classified as “off by 1 letter”, which suggests that the model tends to make broader structural errors rather than near-misses.
- A small number of responses were labeled as “other”, capturing miscellaneous or malformed outputs.

This distribution aligns with prior concerns from Bender & Koller (2020) and Mitchell (2023): that LLMs operate more on surface-level form than abstract symbolic rules, making them vulnerable to failure on rigid, low-level tasks like string manipulation.

```python
def classify_error(row):
    if row["Is Correct"]:
        return "Correct"

    response = row["Response"]
    target = row["Correct Answer"]

    if len(response) != len(target):
        return "Extra/missing letters"

    diff_count = sum(a != b for a, b in zip(response, target))

    if diff_count == 1:
        return "Off by 1 letter"
    elif sorted(response) == sorted(target):
        return "Wrong order"
    else:
        return "Other"

df["Error Type"] = df.apply(classify_error, axis=1)
```

```python
df["Error Type"].value_counts().plot.bar(title="Distribution of Error Types")

plt.ylabel("Count")
plt.xlabel("Error Type")
plt.xticks(rotation=25)
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```



5. Discussion

The results from this project give a view into the way Large Language Models like ChatGPT respond to tiny differences in the wording of prompts, especially when performing symbolic tasks like reversing strings.

Prompt Clarity Matters

The most literal instructions ("Reverse the following string") consistently outscored vague or verbose ones. Prompts A and B, which explicitly instructed the model to reverse or flip the string, scored best (60%), while Prompt D ("Can you tell me.") trailed at 40%. This suggests that precise, imperative language helps LLMs better understand symbolic tasks.

Task Complexity Impacts Accuracy

There was a clear pattern with regards to word complexity and length. Short words like "pan" and "cheese" were flawless, while more complex and longer words like "archetypal" and "subsequently" entirely fell apart for every prompt. This points to the notion that LLMs struggle with character-level manipulation with words that are longer than a point or violate the patterns of usual lexical forms.

Error Types Reveal Form-Based Failures

Most mistakes were either missing or extra letters, or getting things in the wrong order, not simply typos. Quirky to note was that there were no "off by one letter" mistakes, which implies that when GPT is making symbolic tasks error-prone, it is instead falling categorically and not incrementally. This indicates strongly in favor of the argument that GPT isn't really conducting symbolic transformation, but instead is approximating expected outputs via statistical pattern-matching.

Implications for LLM Cognition

These findings substantiate Melanie Mitchell (2023) argument that LLMs rely on heuristics and memorization, not abstract reasoning. The fact that prompt wording can have such extreme influences on success, and the performance collapsing on novel or long strings, suggests GPT-3.5 lacks robust internal representations for symbolic tasks. It performs well when the query is in accordance with patterns in its training set, but not otherwise.

This also aligns with criticism from Bender and Koller (2020), whose argument is that LLMs lack grounded understanding and are operating purely on surface form. Reversal of strings, a cognitively simple but symbolically rigid task, exposes these limits — giving a useful insight into how far LLMs have come, and how far they still have to go in approximating human-like thought.

6. Limitations

While this project gives valuable data concerning prompt sensitivity and symbolic reasoning in LLMs, there are certain limitations that should be mentioned:

- **Small sample size**: The experiment involved only 5 test words and 4 variations of the prompt. A larger, more varied dataset would be required in order to safely extend the results to a wider population.
- **Binary evaluation**: Correctness was measured in a binary fashion. Either entirely correct or not, and did not evaluate partially correct answers (e.g., minor typos or close-reversals). More granular scoring might provide more informative insight into the quality of GPT's output.
- **No context chaining**: All tests were administered one-shot format (i.e., stand-alone prompts). GPT's performance may vary in multi-turn conversation or with examples given in context (few-shot or chain-of-thought prompts).
- **Single model**: Only GPT-3.5 Turbo was tested. Testing other models (e.g., GPT-4, Claude, Gemini) can help in concluding whether these issues are universal across architectures or specific to this version.

Despite these limitations, the results strongly suggest that prompt engineering and task difficulty are central to LLM performance on symbolic reasoning tasks.

7. Conclusion

This project set out to respond to an apparently simple but telling question: Which prompt phrasing leads to the highest accuracy for string reversal tasks in ChatGPT?

The results showed that the way prompts are worded has a big effect on model performance, especially on strict symbolic manipulation tasks. More direct prompts like "Reverse the following string" yielded much higher accuracy than vague or conversational prompts. The model also did not do well with longer and more complex words, showing the constraints of its generalizability across character-level operations.

These findings add weight to the theory that GPT-3.5 Turbo is not thinking like a human but rather relying on statistical associations and pattern-matching, especially when asked to perform something beyond normal natural language generation.

Future Directions

There are a variety of means this research can be expanded:

- Play with more abstract or creative prompt forms, such as metaphors, analogies, or CoT (Chain-of-Thought) forms in order to determine if symbolic reasoning is heightened.
- Attempt more complex multi-step activities like sentence reversal, word encoding in ciphers, or logic manipulations.
- Benchmark across LLMs, including GPT-4, Claude, Gemini, or open-source alternatives like Mistral and LLaMA, to see if newer or differently trained models show more robust behavior.

Tiny exercises like reversing strings serve us ultimately to uncover big questions about how LLMs work, and where the line is between sophisticated mimicry and genuine reasoning.

References

Bender, E. M., & Koller, A. (2020). Climbing towards NLU: On meaning, form, and understanding in the age of data. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 5185–5198. https://doi.org/10.18653/v1/2020.acl-main.463

Mitchell, M. (2023, September 10). *Can large language models reason?* AI Guide. https://aiguide.substack.com/p/can-large-language-models-reason